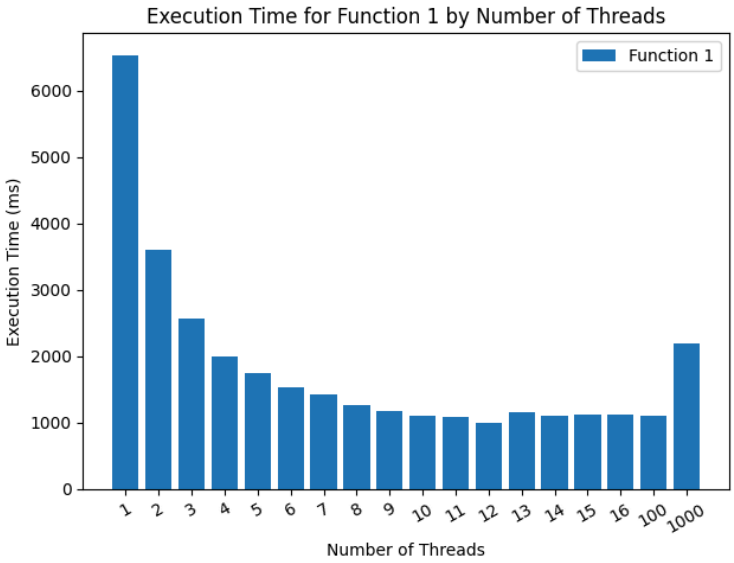
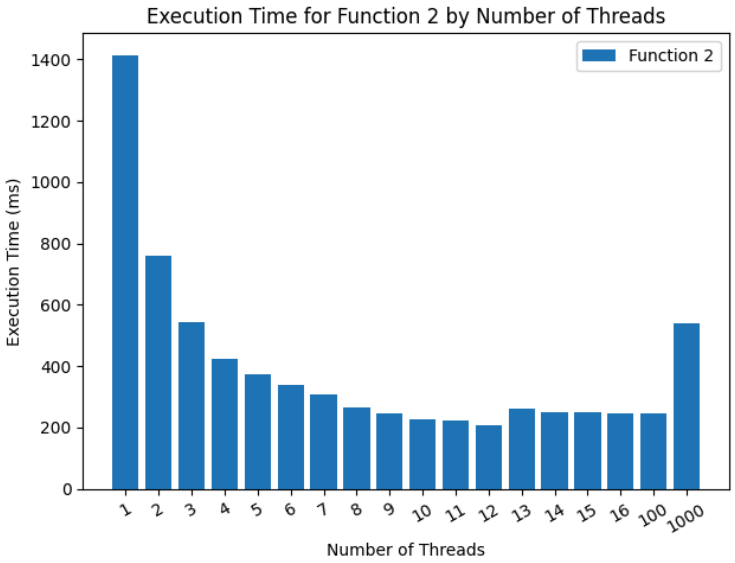
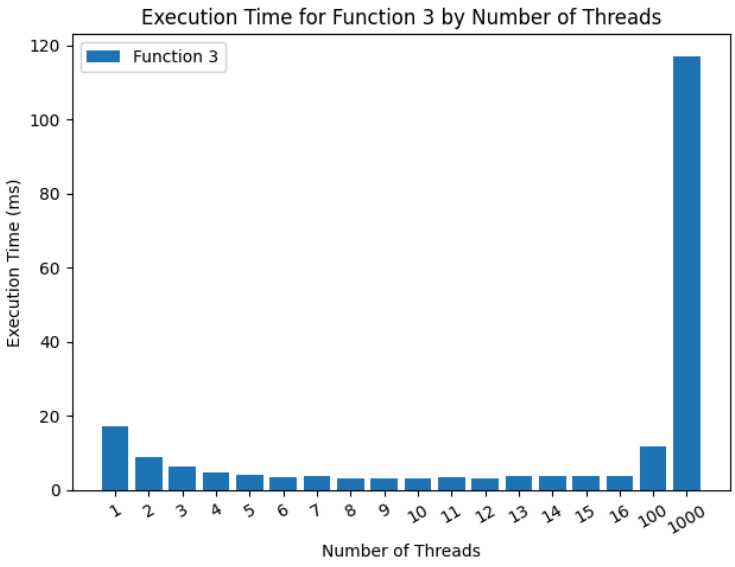
Звіт

Склад команди: Валігурський Антон, Штогрин Олег, Тарнавський Максим-Василь.
Додаткові завдання: 1, 2, 3

Основне завдання







Вищезазначені графіки були побудовані на основі даних, розташованих у файлі “./integral_parallel_measurements/final_measurements.csv”, отриматих після 30 запусків алгоритму обчислення інтеграла для кожного числа потоків. Можемо зробити наступні спостереження:

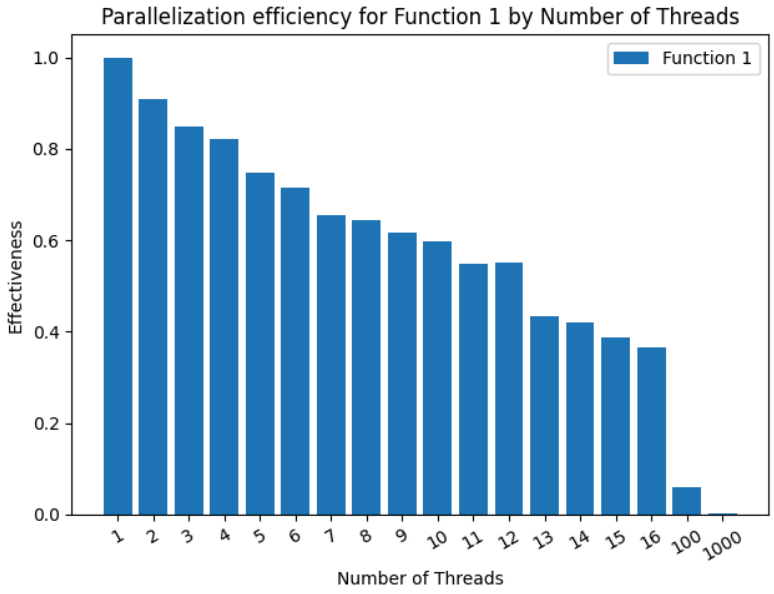
- Очікувано, час роботи алгоритму дуже суттєво зменшувався зі збільшенням кількості потоків від 1 до 12 (12 - кількість логічних процесорів на комп’ютері, з якого відбувалися досліди). Зокрема дуже помітна різниця між послідовним виконанням та двома потоками - час виконання впав ледь не удвічі.

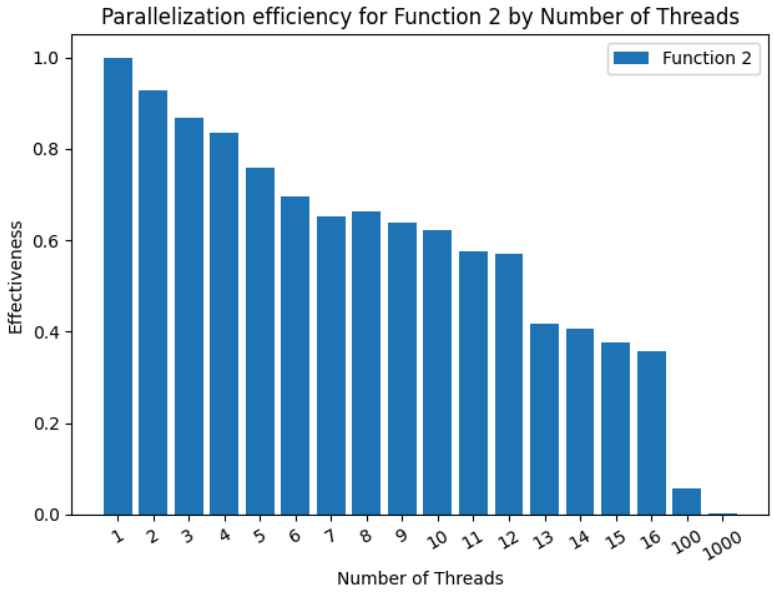
- Цікаво, що час виконання зростав за 13 потоків, а потім знову дещо зменшувався, хоч і не до рівня 12-поточної реалізації - припускаємо, це пов’язано з тим, що 13 - просте число, відповідно цю кількість потоків складніше відносно рівномірно розподілити між логічними процесорами, що призводить до затримок і неузгодженостей. Це припущення підкріплюється тим, що швидкодія також дещо падає навіть за 7 потоків, це можна помітити на графіку 3 функції.

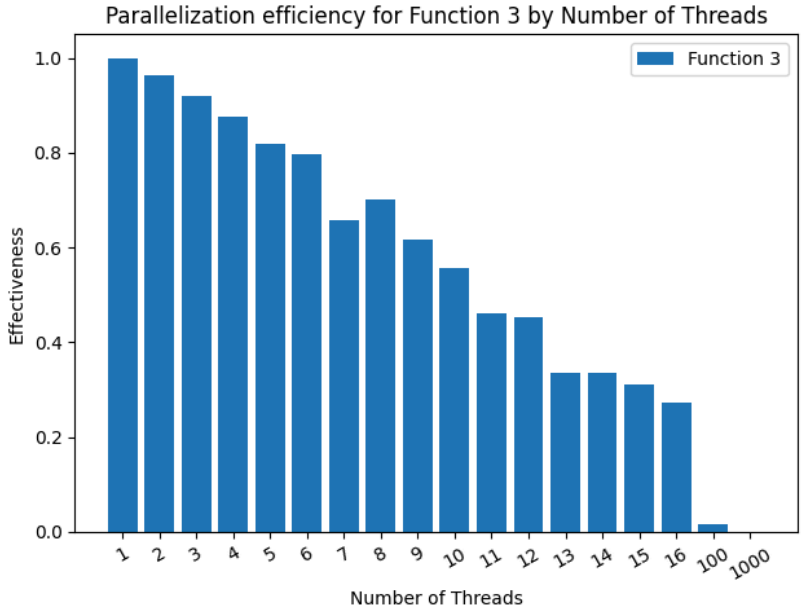
- Врешті, ще більше зростання кількості потоків (100, 1000) призводило лиш до збільшення часу виконання - зокрема це помітно на графіку для 3 функції. Власне обчислення її інтегралу займає всього декілька мілісекунд, тож за цих двох кількостей потоків дуже добре помітне зменшення швидкості, спричинене ними, особливо за 1000 потоків, коли, схоже, робота з ними забирає в декілька разів більше часу, ніж сам алгоритм. Думаємо, це спричинене тим, що коли кількість

потоків переважає кількість логічних ядер, вони починають “змагатися” за ресурси, відповідно система змушена додатково врегульовувати цей процес, зокрема постійно міняти активні потоки на кожному процесорі.

Графіки коефіцієнту ефективності паралелізації



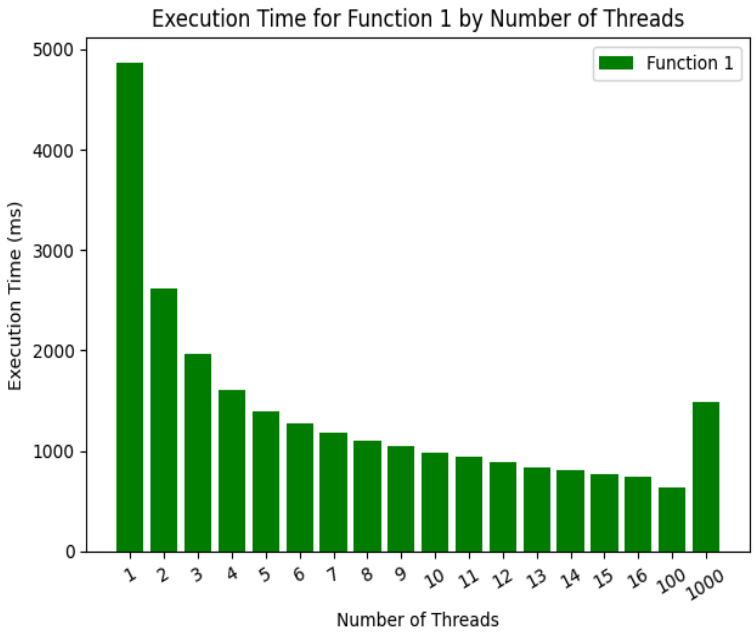


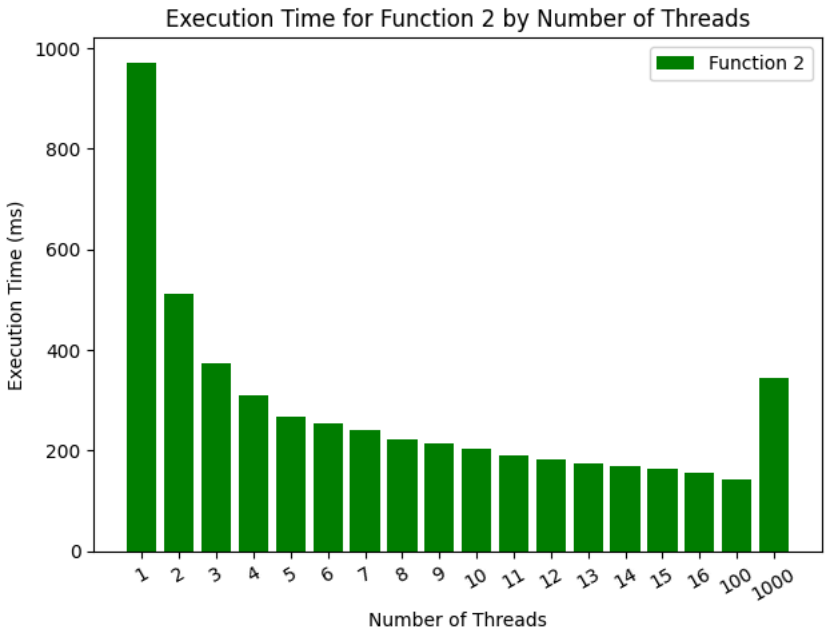


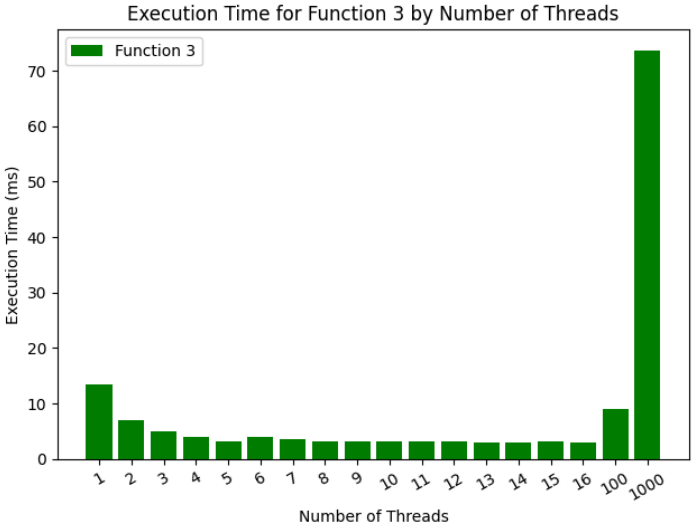
На графіках видно, що ефективність доволі рівномірно падає зі збільшенням кількості потоків, стрімкіше для швидших задач (зокрема обрахунку інтеграла функції 3). Для 1000 потоків вона стає мізерною. Також помітне зменшення ефективності для простих чисел 7 та 13. Загалом ці графіки підкріплюють наші припущення з попереднього пункту.

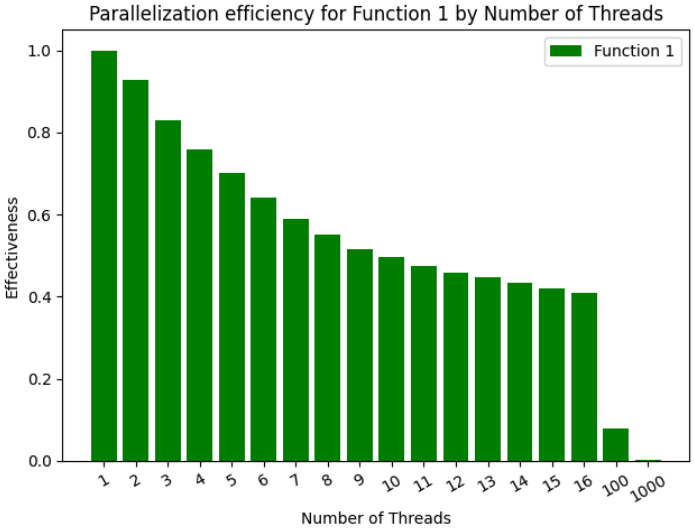
Додаткове завдання 1

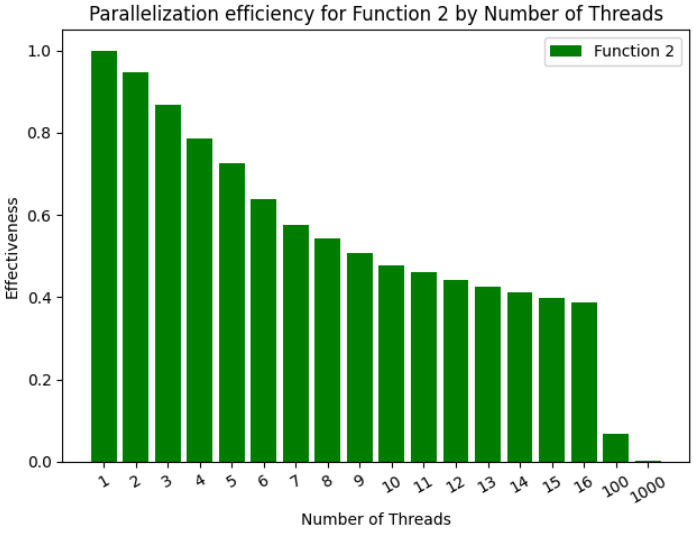
Наступні дані отримані після запуску інтегрування за вводу, ідентичного до вводу в основному звіті, проте на ноутбуці з 14-ма ядрами та 20-ма потоками (враховуючи hyper-threading), виміри розташовані у “./integral_parallel_measurements/measurements_oleg.csv”:

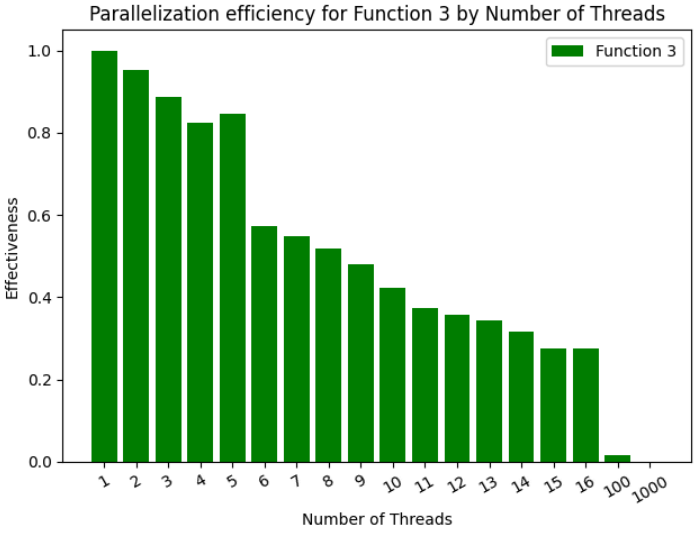












Загальні всновки (вплив кількості потоків на швидкодію, залежність міш швидкодією й кількістю логічних ядер, тенденція зміни ефективності…) справджуються і для цього експерименту, проте є цікаві відмінності:

- Очевидно, за більшої кількості логічних ядер код працює швидше (у нашому випадку, приблизно у 1.4 рази). Також,

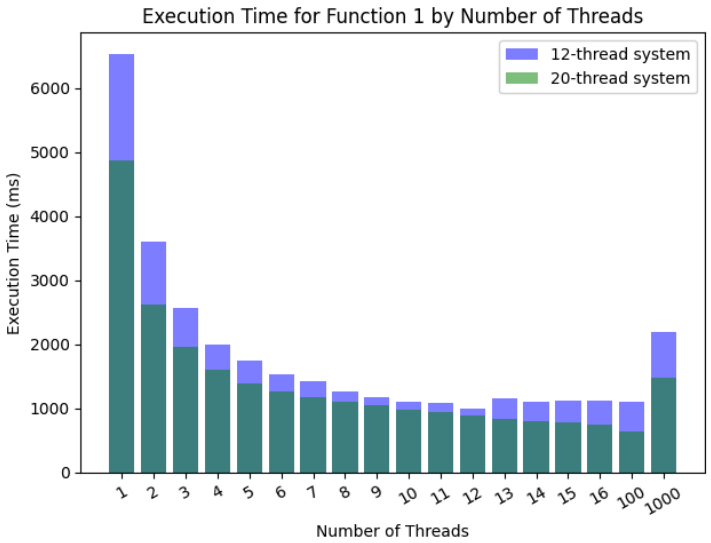
оскільки максимальна кількість логічних ядер більша за 16, немає зростання часу роботи на потоках 13-16.

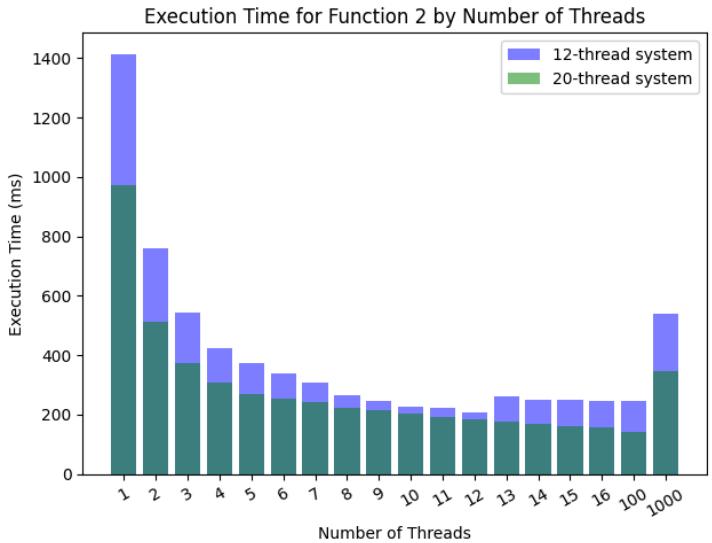
- Цікаво, що зростає час роботи після перевищення 20 потоків теж повільніше - на 100 потоках він досі не повернувся до часу, витраченого на 16-тьох.

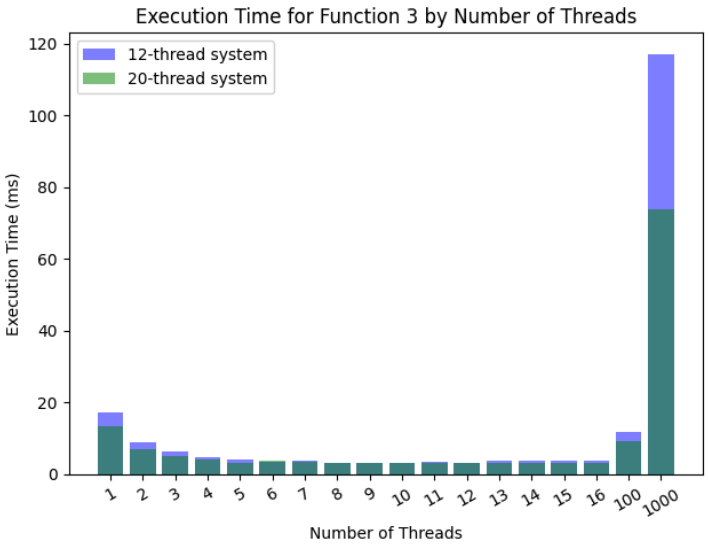
- Ефективність паралелізації також спадає повільніше, тепер графік менш лінійний та навіть став більше нагадувати вітку гіперболи. Цікаво, що на інтервалі 4-12 потоків часто переважає ефективність 12-поточної системи, проте після цієї позначки 20-поточна все ж переганяє її.

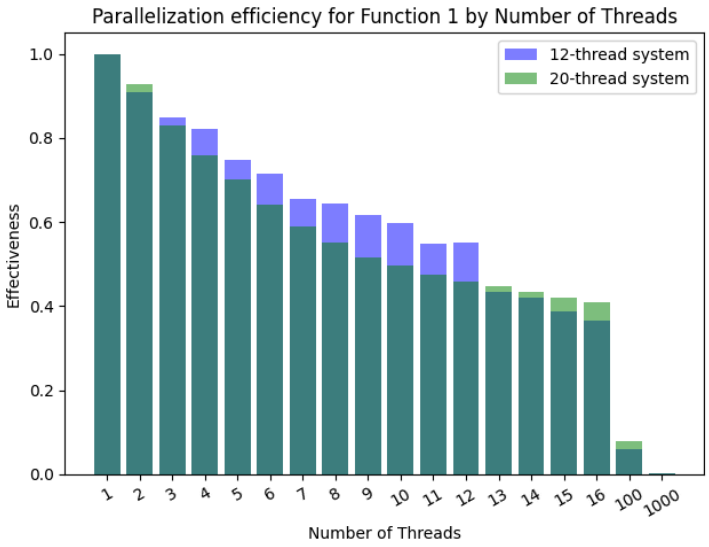
- Зменшення ефективності для 7 та 13 потоків також майже непомітне порівняно з основним експериментом.

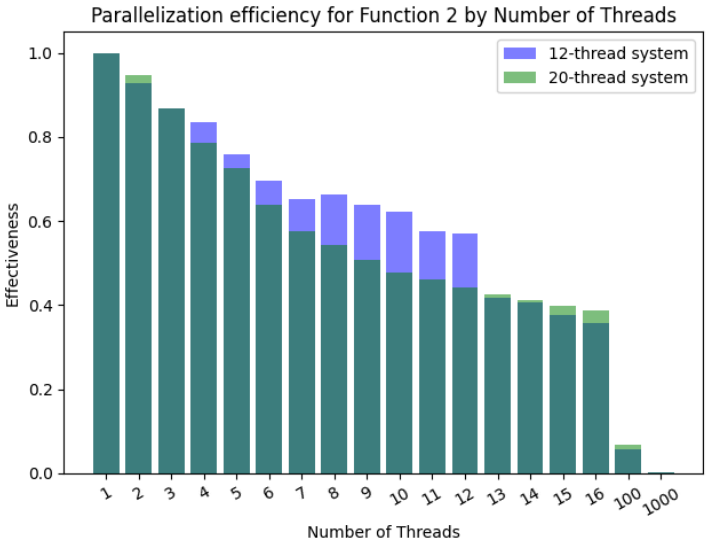
Більш наочне порівняння двох замірів:

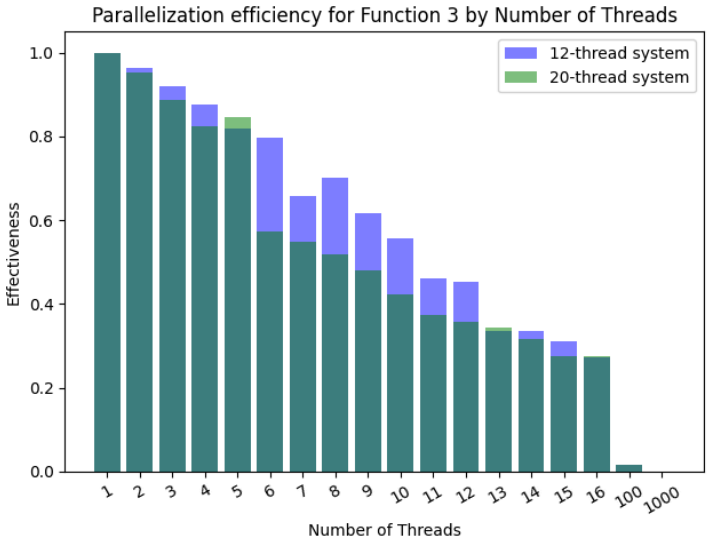





Execution Time for Function 3 by Number of Threads
12-thread system
20-thread system
Execution Time (ms)
Number of Threads


Parallelization efficiency for Function 1 by Number of Threads
12-thread system
20-thread system
Effectiveness
Number of Threads


Parallelization efficiency for Function 2 by Number of Threads
12-thread system
20-thread system
Effectiveness
Number of Threads

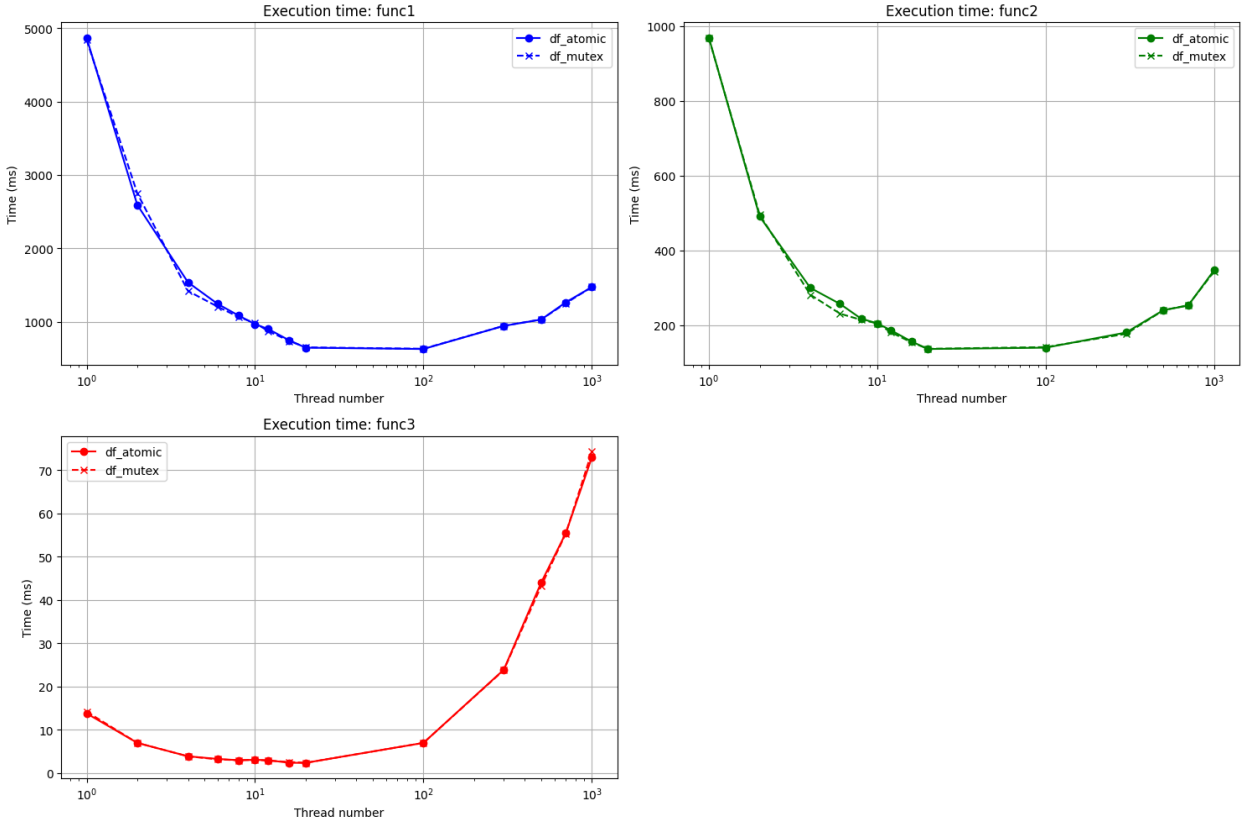

Parallelization efficiency for Function 3 by Number of Threads
12-thread system
20-thread system
Effectiveness
Number of Threads

Додаткове завдання 2

(тестування відбувалося на комп’ютері з 20-ма потоками)

Для додаткового завдання було взято таку кількість потоків: 1, 2, 4, 6, 8, 10, 12, 16, 20, 100, 300, 500, 700, 1000.

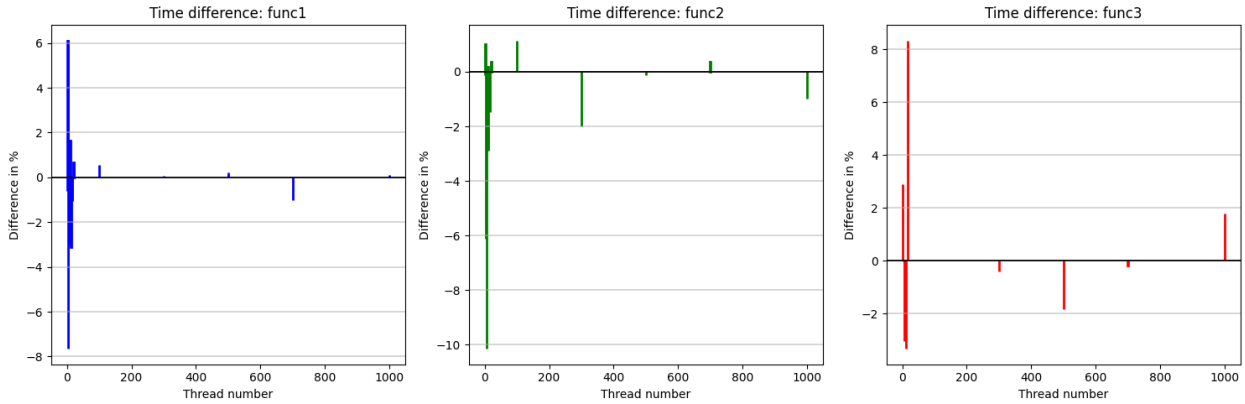
(масштаб є логарифічним)



Як ми можемо спостерігати, відчутної різниці немає між використанням atomic та mutex (хоча atomic в багатьох випадках швидшим за mutex, черех те, що mutex повністю блокує наступні дії іншим потокам). Зрозуміло, що це пов’язано з тим, що в нашій програмі мінімізовано використання що mutex, що atomic: складність обрахування частинки інтегралу складає O(number_x_points*number_y_points*func_complexity), коли ж

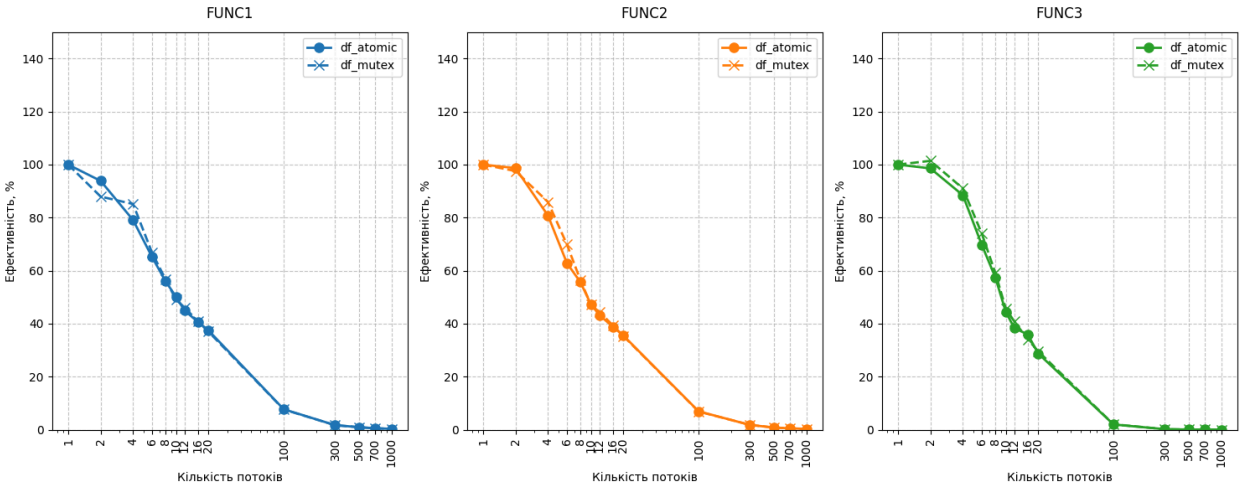
кількість додавань atomic або lock/unlock mutex буде рівна thread_number (зрозуміло, що 1000 lock/unlock mutex буде довше, аніж 1000 додавань в atomic, але в порівнянні з складністю алгоритму, воно займає мізерний час).

Тут можна спостерігати різницю у відсотках, яка рахувалась по формулі для n потоків:
(mutex_function<x>_time - atomic_function<x>_time)
/
atomic_function<x>_time



Можна побачити, що хоч якась різниця спостерігається на кількості потоків до 6, дальше відсоток змін не перевищує двох. В найгіршому випадку різниця досягає 10%.

Якщо ж говорити про коефіцієнт ефективності паралелізму, то він так само показує видиму різницю лиш до 6 потоків.

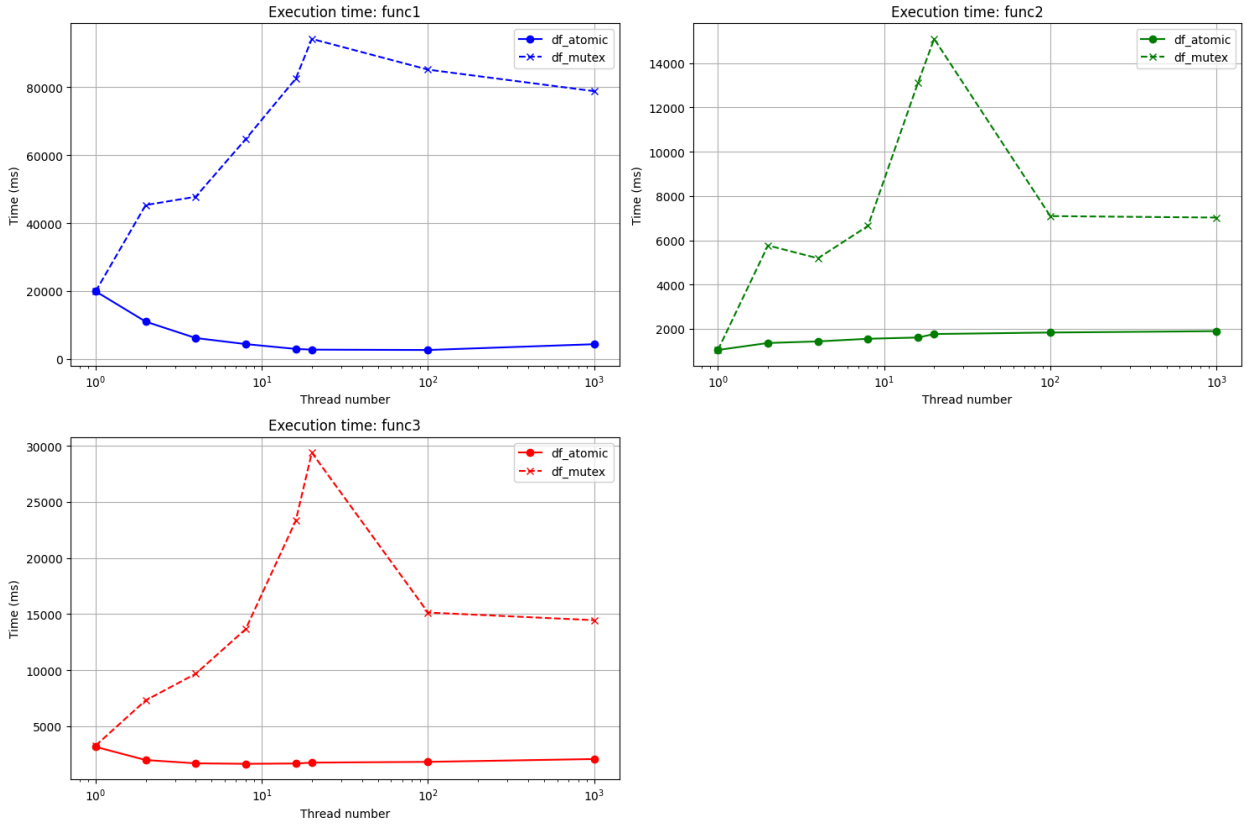


Таким чином дана статистика підтверджує, що при малій кількості використань mutex та atomic, на великій кількості потоків їх ефективність сильно не відрізняється (але потрібно розуміти, що чим більше в програмі буде використано atomic або mutex(як його заміна), тим більша буде між ними різниця).

Додаткове завдання 3

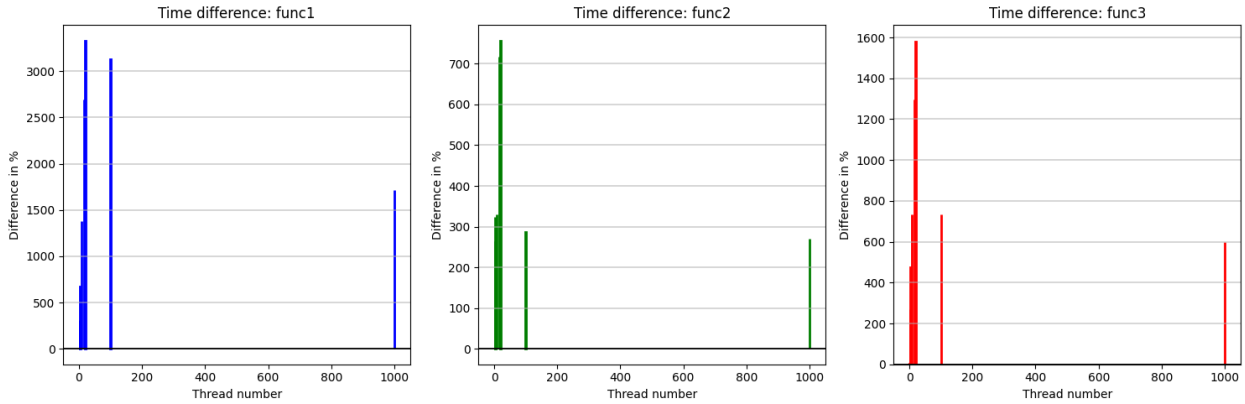
(тестування відбувалося на комп’ютері з 20-ма потоками)

Для додаткового завдання було взято таку кількість потоків: 1, 2, 4, 8, 16, 20, 100, 1000.

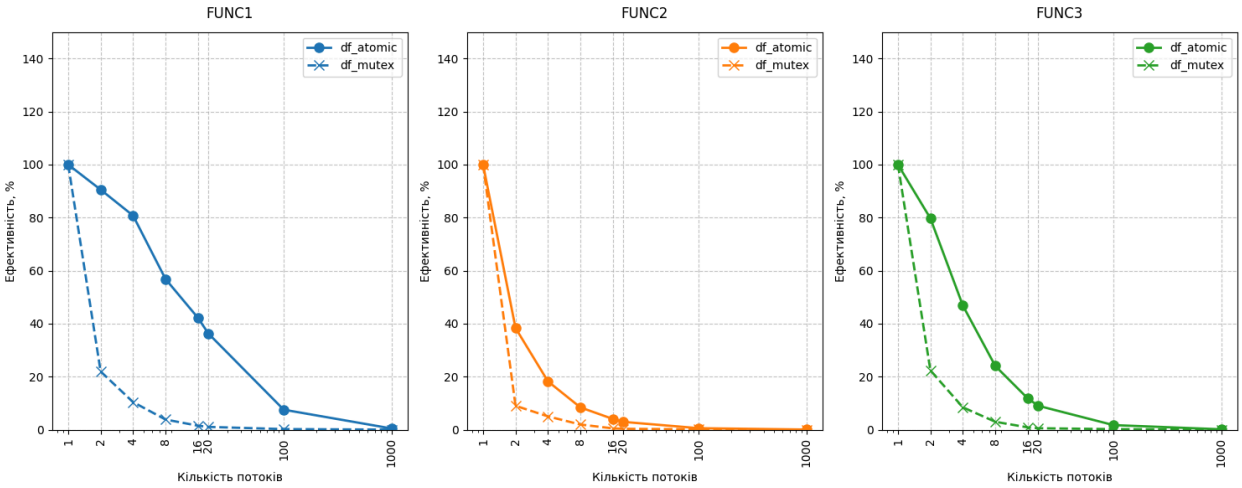


В порівнянні з минулим додатковим завданням, в цьому ми в повній мірі можемо відчути весь жах вкладених м’ютексів. Так як на кожній ітерації викликався м’ютекс, можна здогадатись, що зі збільшенням кількості потоків, підрахунок інтегралу буде ставати все повільніше і повільніше, через те, що як такої паралельності практично немає, бо кожне додавання і обрахування функції відбувається послідовно, додатково до цього накладається ще величезна кількість lock і unlock м’ютексів. На диво, після піку в 20 потоках, можна помітити спад в 100 і 1000 потоках, це, швидше за

все, пов’язано з тим, що йде постійне переключення між потоками, і так як не всі з них можуть постійно ‘застоюватись’ на м’ютексі (умовно поки у нас виконується робота в м’ютексі, тільки умовні 300 потоків з 1000 доходять до нього, коли ще 700 лиш доходять до цього місця в програмі), програма працює дещо ефективніше, ніж у випадку, коли всі потоки стоять і чекають виконання обрахування операції в м’ютексі. Неінтуїтивно, але саме в такій ситуації з більшою кількістю потоків (більшою за фізично доступну) програма працює швидше.



Як ми бачимо, вкладені м’ютекси в певних випадках можуть перевищувати різницю у відсотках з атоміками аж в 3000%! Що явно показує, чому вкладені м’ютекси, в певних ситуаціях, гірше простого послідовного коду.



З очікуваного, ефективність атоміка виявилась, щонайменше, значно кращою в порівнянні з м’ютексом. Нічого дивного немає, ефективності від такого паралелізму практично 0.

Загальним висновком буде: не робити вкладених м’ютексів і атоміків. Порівнюючи з додатковим 2, ми бачимо, що час виріс в кілька разів у всіх способах, а особливо у вкладених м’ютексів. Ефективність паралельності ж впала значно.